

20250702 日报

今日工作内容

1、学习了解了 virtual actor (grain) 的设计原理，并比较了它与传统 actor 的不同。

virtual actor 是基于 actor 模型的进一步发展，它具有唯一标识 ID，其生命周期由框架管理，无需显式创建或销毁。服务不需要知道 actor 的物理位置，只需要用唯一标识即可，即位置透明。第一次向其发送消息时被自动激活并放置在集群节点上（不管被发送消息的 actor 是否存在），长时间未使用则从内存卸载，但状态仍可持久化存储，需要时能重新激活。

virtual actor 与传统 actor 的最主要的差异：

- 生命周期管理
 - 传统 actor 模型需要显式创建与销毁；
 - virtual actor 是按需激活与自动回收，在首次收到消息时自动激活 actor 并加载状态；闲置超时后会自动停用并持久化状态，释放内存资源。开发者无需干预生命周期；
- 位置透明性
 - 传统 actor 模型中调用方需知晓目标 actor 的物理节点位置 (ip+port)，并且在节点故障时需开发者实现 actor 迁移逻辑；
 - virtual actor 通过唯一逻辑 id 寻址，运行时自动路由至目标节点，并且节点故障时，会自动将 actor 迁移至健康节点；

virtual actor 模型中不需要显式的创建 actor。在第一次向 actor 发送消息时，actor 被隐式激活并放置在集群中的一个节点上，因此对于 virtual actor 模型是不需要显式创建 actor 的。当一个 actor 在一段时间内未被使用或不执行操作时，actor 将以静默方式从内存中移除，如果一个节点发生故障，会自动将 actor 转移到监控节点。

2、误操作需求中 itemchange 和 acntbuy 字段处理失败的告警点上传。

3、误操作需求中在高并发消费 kafka 消息而调用服务时的限流处理：从云彤哥代码中学习了协程池的简单创建。

```

// 基于"github.com/panjf2000/ants"中的 ants 包直接创建协程池

var gConsumerPool *ants.Pool

func CreatePool() error {
    var err error

    //创建 Consumer 协程池
    consumerPoolCnt := rainbow.GetConfig().ConsumerPoolCnt
    gConsumerPool, err = ants.NewPool(int(consumerPoolCnt))
    if err != nil{
        return err
    }

    log.Infof("Create gConsumerPool Pool, cnt %v",
consumerPoolCnt)

    return nil
}

func GetConsumerPool() *ants.Pool {
    return gConsumerPool
}

```

单纯只是暴力设计一个默认大小为 2000 的固定协程池，没有其他管理机制（如动态扩缩容、流量控制等），它就是单纯的限制最大并发任务数，避免无限制创建协程导致资源耗尽，当瞬时请求量超过 2000 时，新任务会阻塞在任务队列中，而非直接创建新协程，从而起到了应对流量陡增的处理能力。

今日所遇问题：

1. 对于误操作需求，从 kafka 消费的时候会瞬间有大量并发请求，因此需要考虑服务限流的问题，当前暂时考虑的处理就是利用协程池；
2. 主逻辑出错时应该添加告警点，如 `metrics.IncrCounter("acnt_buy-kafka_handle_failed", 1)`。

明日待办：

1. 和强哥、岳哥商量下一步需要做的事情，如果是 `virtual actor`，那么就落实具体需求。

